

CoAct-1: 具备编码动作的计算机使用多智能体系统

CoAct-1: COMPUTER-USING MULTI-AGENT SYSTEM WITH CODING ACTIONS

Linxin Song^{1*}, Yutong Dai², Viraj Prabhu², Jieyu Zhang³, Taiwei Shi¹, Li Li¹, Junnan Li²
Silvio Savarese², Zeyuan Chen², Jieyu Zhao¹, Ran Xu², Caiming Xiong²

¹ 南加州大学 ² Salesforce ³ 华盛顿大学

代码: <https://github.com/SalesforceAIResearch/CoAct-1>

摘要

通过图形用户界面 (GUI) 操作计算机的自主智能体, 在复杂的长时程任务上往往难以兼顾效率与可靠性。为这类智能体加入规划器虽然能够改善任务分解, 但若所有动作仍必须通过 GUI 操作完成, 系统依然受制于这种交互方式固有的局限, 因而脆弱且低效。本文提出一种更稳健、更灵活的范式: 让智能体把编码作为一种增强动作。我们提出 CoAct-1, 这是一种将 GUI 控制与直接程序化执行协同结合的新型多智能体系统。CoAct-1 包含一个编排器 (Orchestrator), 它把子任务动态委派给传统的 GUI 操作员 (GUI Operator) 或专门的程序员 (Programmer) 智能体; 后者可以编写并执行 Python 或 Bash 脚本。这种混合方法使智能体能够在文件管理、数据处理等任务中绕过低效的 GUI 动作序列, 同时在必要时继续使用视觉交互。我们在具有挑战性的 OSWorld 和 WindowsAgentArena 基准上评估该系统; CoAct-1 在 OSWorld 上取得 60.8% 的成功率, 在 WindowsAgentArena 上取得 52.5% 的成功率, 均达到新的最先进水平并显著优于既有方法。此外, 该方法显著提升了效率: 在 OSWorld 上, 完成任务所需的平均步数降至 10.15, 而领先 GUI 智能体需要 15 步。结果表明, 将编码整合为核心动作, 为通用计算机自动化提供了一条能力更强、效率更高且更具可扩展性的路径。

1 引言

近期关于计算机使用智能体的进展, 主要聚焦于通过图形用户界面 (GUI) 进行操作。由视觉语言 (动作) 模型驱动的 GUI 智能体 (Li et al., 2023; Deepmind, 2025a;b; OpenAI, 2024; 2025a; Qin et al., 2025; Xie et al., 2025a; Yang et al., 2025) 已经展现出完成多类任务的能力, 但在长时程规划以及 GUI 元素密集环境中的交互方面仍常常遇到困难。例如, Office 生产力软件中的日常任务往往涉及漫长而复杂的精确 GUI 操作序列: 先在含多个工作表的电子表格中定位特定表格, 再按复杂条件筛选、复制结果, 并将其保存为新的 CSV 文件。类似地, 在嵌套目录结构中查找所有图像文件、把它们缩放至指定尺寸、再将整个目录压缩为单个归档文件; 若依靠点击、拖拽等 GUI 动作完成, 同样既脆弱又低效。在这些场景中, 现有智能体经常受到视觉定位歧义 (例如混淆外观相似的图标或菜单项) 以及长期交互中错误概率累积的影响。一次误点或对某个界面元素的误解, 就可能使整个任务偏离轨道。

为应对这些挑战, 一条重要研究路线是为 GUI 智能体配备专门的高层规划器。GTA-1 (Yang et al., 2025) 等方法及其他模块化系统 (Yang et al., 2024; Xu et al., 2024; Agashe et al., 2024;

*该工作完成于 Salesforce 实习期间。

2025), 使用 OpenAI o3 (OpenAI, 2025b) 之类的强大语言模型, 将用户的高层目标分解为一系列更易处理的子任务。这样的层次化分解通过提供结构化计划, 可以改善复杂多步问题上的表现。然而, 该范式并未从根本上解决完全依赖 GUI 执行所带来的低效和脆弱性。即使已经完成高层规划, 智能体仍需浏览菜单、点击按钮并向字段输入内容; 即便某些操作本可通过程序化手段更直接、更可靠地完成, 也不例外。因此, 系统仍容易受到规划不确定性、视觉感知错误, 以及高层规划与底层动作生成之间整合难题的影响。

本文主张并实现一种更灵活、更强大的动作空间。我们提出一种混合方法, 把 GUI 操作直观、类人的优势, 与通过代码直接同系统交互所具备的精确性、可靠性和效率结合起来。我们提出 **CoAct-1** (Computer-using Multi-agent System with **C**oding **A**ctions), 它由编排器、程序员和 GUI 操作员三个专用智能体组成。高层编排器充当中央规划器, 负责分解用户目标并为每个子任务确定合适的动作模态。根据分析结果, 它将任务交给两类不同的执行智能体之一: 程序员负责为文件管理、数据处理或环境配置等后端操作编写并执行 Python 或 Bash 脚本; 基于视觉语言模型 (VLM) 的 GUI 操作员则负责点击按钮、浏览可视界面等前端动作。通过这种动态委派, CoAct-1 可以在适当策略性地跳过低效 GUI 序列, 改用稳健的单次代码执行, 同时仍可针对需要视觉操作的任务利用视觉交互。

实验分析为这种混合设计的优势提供了有力证据。在 OSWorld 和 WindowsAgentArena 基准上, CoAct-1 分别取得 60.76% 和 52.50% 的总体成功率, 刷新了最先进水平。在 OSWorld 上, 这相较 Agent S2.5 (55.98%) 等领先基线有显著提升。在程序化控制优势明显的任务类别中, 性能增益最为突出。例如, 在 Calc (70.21%)、多应用 (47.88%) 和 VS Code (78.26%) 任务中, 程序员执行精确脚本的能力, 使系统相较最强的纯 GUI 方法获得大幅提升。除提高成功率之外, 我们的双模态方法还显著改善了操作效率。通过用简洁代码替代漫长且易错的点击序列, CoAct-1 在 OSWorld 上平均仅需 10.15 步即可解决任务, 与 GTA-1 等智能体所需的 15 步形成鲜明对比。这种效率说明, 本方法有望为通用计算机自动化铺就一条更稳健、可扩展的路径。

2 相关工作

屏幕解析与视觉定位。第一类工作聚焦于直接从像素中感知和定位 GUI 元素, 而不依赖 DOM 或无障碍接口。*OmniParser* 学习用于纯视觉理解的屏幕解析基元 (Lu et al., 2024)。在定位方面, *SeeClick* (从指令定位目标)、*Aria-UI* (面向 GUI 的指令定位) 和 *UGround* (通用 GUI 定位) 把语言映射到屏幕上可执行操作的位置 (Cheng et al., 2024; Yang et al., 2024; Gou et al., 2024)。*OS-Atlas* 训练一种基础动作模型, 使其能够泛化到多样化界面 (Wu et al., 2024)。*ScreenSpot-Pro* 等专门的定位评测, 还进一步衡量了专业高分辨率环境中的定位能力 (Li et al., 2025)。

原生端到端 GUI 智能体。另一条路线训练在单一模型中统一感知、推理与动作的原生智能体。*UI-TARS* 和 *OpenCUA* 是这种方法的代表: 它们使用跨应用的统一鼠标/键盘动作空间, 而不采用手工设计的控制器 (Qin et al., 2025; Wang et al., 2025a;b)。*AGUVIS* 则进一步探索能够跨界面泛化的统一纯视觉 GUI 智能体 (Xu et al., 2024)。

模块化规划器-定位器智能体。第二类研究明确区分做什么与在屏幕何处、如何操作: 语言规划器提出子目标, 视觉模型则定位每一步动作。代表性系统包括 *SeeClick* 和 *OS-Atlas* (Cheng et al., 2024; Wu et al., 2024)。*GTA-1* 通过测试时扩展强化这种两阶段范式: 采样多个候选动作, 再使用多模态大语言模型 (MLLM) 裁判从中选择, 从而提高系统在高分辨率、元素密集 GUI 上的稳健性 (Yang et al., 2025)。*Agent-S / Agent-S2* 与 *AutoGen* 等其他相关开放框架,

则为多智能体编排和工具调用提供了可复用基础设施 (Agashe et al., 2024; 2025; Song et al., 2025a; Zhang et al., 2024; Wu et al., 2023; Zhang et al., 2023; 2025b)。

混合智能体框架。在纯 GUI 交互之外，一些智能体系统会在运行时动态组合工具和 API 以扩展能力。例如 *UFO-2* (Zhang et al., 2025a)、*PyVision* (Zhao et al., 2025)、*BeyondBrowsing* (Song et al., 2025b) 和 *ALITA* (Qiu et al., 2025)。这些系统虽然不局限于 GUI/计算机使用智能体 (CUA)，但都遵循动态构造并调用工具的原则。

3 以编码为动作的计算机使用智能体

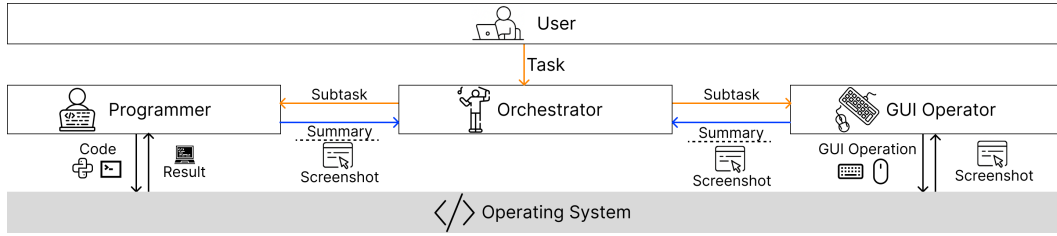


图 1: CoAct-1 的多智能体系统设计。系统包括：作为高层规划器、负责分解目标并把子任务委派给合适执行智能体的**编排器**；通过多轮编码与操作系统交互的**程序员**；以及利用视觉语言能力执行可视界面动作的**GUI 操作员**。

本文引入一种新的系统交互动作——编码——来替代部分冗余且脆弱的 GUI 动作。不同于为每个应用或网站归纳 API/SDK，我们关注的是让智能体在强语言模型引导下进行自由形式编码，以解决计算机使用问题。具体而言，我们设计了多智能体系统 CoAct-1，并引入一个能够通过“编码-观测”循环与操作系统交互的新智能体——程序员。编排器充当高层控制器，决定把子任务交给程序员还是 GUI 操作员。整体框架如图 1 所示。

3.1 问题定义

我们把通用计算机控制形式化为一个交互式决策过程。在每个时间步 t ，智能体观测主要由屏幕截图构成的计算机环境 $o_t \in \mathcal{O}$ ，并依据策略 $\pi(a_t | H_t, G)$ 采取动作 $a_t \in \mathcal{A}$ 。其中， $H_t = (o_1, a_1, \dots, o_{t-1}, a_{t-1}, o_t)$ 表示历史上下文， G 是用户以自然语言提供的高层目标。当动作空间 $\mathcal{A}$ 被限制为底层 GUI 操作时，学习有效策略尤其困难。管理嵌套文件、处理电子表格数据等复杂任务，可能需要漫长而繁复的 GUI 动作序列，因而效率低下，并极易发生错误传播：一次误操作就可能使整个任务失败。

为解决这一局限，我们引入整合直接程序化控制的混合动作空间。它在标准 GUI 动作空间基础上扩展而来，记为 $\mathcal{A} = \mathcal{A}_{\text{GUI}} \cup \mathcal{A}_{\text{Code}}$ 。动作 $a_t \in \mathcal{A}_{\text{GUI}}$ 表示对图形界面的直接操作（例如鼠标点击、键盘输入）；相较之下，动作 $a_t \in \mathcal{A}_{\text{Code}}$ 是一段直接与操作系统后端交互的 Python 或 Bash 脚本。这使智能体能够用一个稳健步骤完成文件操作、数据处理等复杂任务，从而有效绕过脆弱低效的 GUI 序列。在 CoAct-1 中，策略 π 采用层次化实现：高层编排器充当元策略 π_{orch} ，分析当前子任务，并将其委派给两个专用执行策略之一——GUI 操作员以 π_{GUI} 实现 $\mathcal{A}_{\text{GUI}}$ 中的动作，程序员则以 π_{Code} 实现 $\mathcal{A}_{\text{Code}}$ 中的动作。

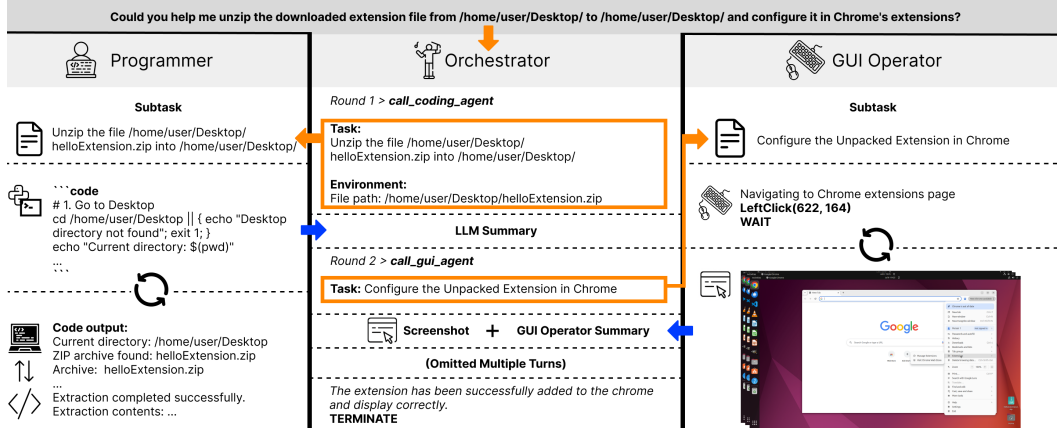


图 2: CoAct-1 工作流程示意图。对于给定用户任务，编排器可以选择调用程序员或 GUI 操作员解决子任务。程序员通过编码与操作系统交互，GUI 操作员则通过 GUI 操作与系统交互。

3.2 面向计算机使用的多智能体系统设计

我们的多智能体系统，是问题定义中层次化策略 π 的架构实例。系统包含编排器、程序员和 GUI 操作员三个专用智能体，它们协作生成动作轨迹 τ ，以完成用户目标 G 。每个智能体都建立独立对话来履行各自职责。

编排器。编排器实现高层元策略 π_{Orch} ，依据完整观测历史 H_t 和总体目标 G 负责任务分解与动态规划。编排器不直接同操作系统交互；它的主要职能，是在专用子策略 π_{Code} 和 π_{GUI} 中选择最适合执行当前子任务的一项。子任务完成后，编排器会收到执行过程摘要和新观测 o_{t+1} （一张屏幕截图，以及反映当前系统状态的摘要），并以此支持下一步决策。若判断总体目标 G 已经完成，它便输出终止信号。

程序员。程序员实现专用策略 π_{Code} ，负责生成动作 $a_t \in \mathcal{A}_{\text{Code}}$ 。收到编排器委派的子任务后，它会与代码解释器展开多轮对话，生成由解释器执行的 Python 或 Bash 脚本。由代码执行结果构成的反馈，使程序员能够反思并改进代码，直至子任务完成。编排器会向程序员提供足够的上下文，例如根据 H_t 推断出的文件路径或窗口信息，从而使其代码生成有明确依据。

GUI 操作员。GUI 智能体是一个视觉语言动作模型，它实现基于 GUI 的策略 π_{GUI} ，用于生成动作 $a_t \in \mathcal{A}_{\text{GUI}}$ 。与程序员类似，GUI 操作员通过多轮交互循环完成被委派的子任务。在“感知-动作”循环的每一步中，智能体以当前屏幕截图和子任务指令为输入，生成一个 GUI 动作（例如鼠标点击或键盘输入）。GUI 动作解释器在操作系统上执行该动作，操作系统随即提供新的屏幕截图作为观测反馈。GUI 操作员根据新的视觉信息决定后续动作，循环往复，直到子任务完成。

3.3 workflow 与记忆设计

CoAct-1 所实现的层次化策略，需要一个结构化 workflow 和记忆系统来管理智能体之间的信息流。该设计确保每个智能体都拥有必要上下文，同时不会被任务其他部分的无关细节淹没。整体 workflow 如图 2 所示。

workflow。 workflow 开始时，充当元策略 π_{Orch} 的编排器将子任务委派给合适的执行智能体，即程序员或 GUI 操作员。选中的智能体随后进入自己的多轮交互循环以解决子任务，并产生记录其动作与环境响应的详细对话历史。子任务完成后，专用摘要模型处理这段详细历史，

把整个交互压缩为简洁摘要，其中涵盖关键动作和最终结果。该摘要连同表示环境新状态的最终屏幕截图一起返回编排器。这一交接机制为编排器提供了对总体任务历史 H_t 的提炼高层更新。

面向任务分解的记忆设计。 CoAct-1 采用层次化且彼此隔离的记忆结构：

- **编排器记忆：**编排器维护长期主记忆，对应问题定义中的历史上下文 H_t 。它包含初始用户目标 G ，以及每个子任务完成后从执行智能体接收的摘要与屏幕截图序列。汇总后的历史为所有高层规划决策提供上下文。
- **执行器记忆：**程序员和 GUI 操作员各自维护短期工作记忆，只在其所负责子任务持续期间保持活动。该记忆包含它们与操作系统多轮交互的“实例对话历史”。

为保证模块化和专注性，这些记忆彼此隔离；各智能体不会直接共享对话历史。此外，一旦执行器完成子任务并向编排器汇报，其工作记忆便会被清空。这种重置机制十分关键，因为它使执行智能体能够完全专注于新收到子任务的上下文，不受先前无关交互影响。

4 实验

4.1 基准数据集

我们在 OSWorld (Xie et al., 2024) 和 WindowsAgentArena (Bonatti et al., 2024) 上评估 CoAct-1。二者都是可扩展的真实计算机测试平台，通过像素流和操作系统 Shell 接口向智能体开放 Windows 或 Ubuntu 操作系统。OSWorld 包含 369 个任务，WindowsAgentArena 包含 154 个任务。这些任务覆盖常用生产力工具、集成开发环境、浏览器、文件管理器和多应用工作流，因而会同时考验智能体在异构 GUI 环境中的视觉语言定位与长时程规划能力。

4.2 基线

我们将 CoAct-1 与两类计算机使用智能体进行比较：端到端模型和智能体方法。这些基线代表了基于 GUI 的任务自动化前沿。

端到端模型。 端到端模型以用户指令和操作系统屏幕截图为输入，不使用任何智能体工作流，仅通过模型推理输出相应动作。*OpenAI o3* (OpenAI, 2025b) 是 OpenAI 的前沿推理模型，擅长多步问题求解以及多用途、上下文感知的辅助。*OpenAI CUA 4o* (OpenAI, 2025a) 运用视觉和推理同图形用户界面交互，控制鼠标和键盘执行任务；Operator、ChatGPT agent 等服务背后使用了这项技术。*UI-TARS* (Qin et al., 2025) 提出完全端到端、仅依赖屏幕截图的原生 GUI 智能体模型，在单一模型中统一感知、推理、记忆与动作。*OpenCUA (32B)* (Wang et al., 2025b) 则提出完全开源的框架，包含可扩展数据收集工具、首个大规模跨操作系统计算机使用任务数据集、反思式思维链推理流水线，以及强大的视觉语言智能体模型。

智能体方法。 智能体方法包括规划器-定位器、规划器-多定位器等不同结构的单智能体与多智能体系统。这些基线主要通过增强语言模型的底层定位能力来改善计算机操作表现。*Jedi-7B w/ o3* (Xie et al., 2025b)：本文中的 Jedi 指在 Jedi 数据集上训练的 Qwen2.5-VL。其作者将 Jedi 接入智能体栈，把高层计划转化为像素级精确的 GUI 动作，并在 OSWorld、WindowsAgentArena 和多个定位基准上取得大幅提升。*GTA-1 w/ o3* (Yang et al., 2025)：GUI 测试时扩展智能体 GTA-1 面向高分辨率界面中的规划歧义和动作定位难题；它采用“测试时扩展”策略，生成多个可能动作，再用裁判模型选出最优动作，并使用 GRPO 训练强定位器。*Agent*

S2.5 w/o3 (Agashe et al., 2025): Agent S2.5 是组合式规划器-多定位器框架。规划器生成高层次子目标，多个定位器执行这些目标，并通过定位专家混合机制把 GUI 元素定位委派给视觉、文本和结构专家；两个层级在每个子目标之后都会主动重新规划，以适应不断变化的屏幕。

除上述强基线外，我们还在 WindowsAgentArena 上加入 Agent S (Agashe et al., 2024) 和 NAVI (Bonatti et al., 2024) 作为基线。

4.3 实现细节

环境。我们在 Linux 上测试 CoAct-1，并扩展了 OSWorld 的 RESTful 服务器。具体而言，我们实现了一个远程代码解释器，它可以接收较长的 Python 和 Bash 脚本，并把执行结果返回给发送方。另一方面，对于每个任务，OSWorld 都会建立初始状态，例如打开一组应用或特定网站，或者把指定文件下载到指定位置。初始状态就绪后，我们截取屏幕截图，并将其与用户任务一起作为 CoAct-1 和各基线的初始输入。

CoAct-1 设置。我们使用 AG2 (Wu et al., 2023) 实现 CoAct-1。其中，编排器采用 OpenAI o3，程序员采用 OpenAI o4-mini。GUI 操作员的骨干模型为 OpenAI computer-use-preview，这是一种由 OpenAI 针对计算机使用微调的视觉语言动作模型。我们还使用 o4-mini 作为摘要器，对程序员与编排器之间的对话历史进行总结。程序员的最大轮数 I 设为 20，GUI 操作员的最大步数 K 设为 25，编排器的最大轮数 J 设为 15。因此，CoAct-1 的系统交互次数（即步数）上限为 375；但如图 3d 所示，所有样例都会在 150 步之前提前停止。更多细节见 Appendix C。

评估协议。我们使用 OSWorld 和 WindowsAgentArena 提供的基于规则的评估器评估方法。每个评估器在内部都表示为一个布尔表达式，由基准作者手工设计的 134 个基于实际执行的原子评估器构成。对于给定任务，基准使用逻辑 AND/OR 运算符组合这些原子条件；例如，“通过”可能要求（文件已导出 AND MD5 匹配）AND（邮件已发送 == True）。

4.4 结果

表 1 和表 2 所列实验结果明确表明，CoAct-1 在两个具有挑战性的真实计算机操作基准上均达到新的最先进水平。这些发现验证了我们的核心假设：在传统 GUI 操作之外整合程序化动作，可以形成更稳健、更高效且更具泛化能力的计算机自动化范式。

OSWorld 上的表现。在综合性 OSWorld 基准（表 1）上，CoAct-1 展现出更优的性能与效率。在 150 步限制内，它最终取得 60.76% 的成功率，显著领先 Agent S2.5 w/o3 (55.98%) 和 GTA-1 w/o3 (53.07%) 等当前最强智能体框架。混合架构的优势不仅体现在峰值表现，也体现在面对不同任务复杂度时的一致性：在 100 步时，CoAct-1 已以 59.93% 的成功率领先，并超过所有其他基线的最终成绩。在程序化控制最有效的类别中，本方法优势尤其明显。对于需要复杂数据或文件系统交互的任务，CoAct-1 在办公、操作系统和多应用类别上分别达到 64.80%、75.00% 和 47.87%。这些领域历来是纯 GUI 智能体较为脆弱之处；上述出色表现凸显了把后端操作委派给程序员智能体的有效性，因为它可以针对电子表格、文件操作和跨应用数据流执行精确可靠的脚本。

WindowsAgentArena 上的表现。我们进一步在 WindowsAgentArena 基准（表 2）上评估 CoAct-1。结果显示，该框架能够把能力成功迁移到另一种操作系统，并再次以较大优势达到最先进水平。CoAct-1 的总体成功率为 52.5%，相较 Agent S2 (29.8%) 和 NAVI (19.5%) 等先前领先方法有显著提升。WindowsAgentArena 上的分类结果进一步支持了我们的核心主张：CoAct-1 在系统级和工具类任务上优势突出，Windows 系统与 Windows 工具类别分别达

表 1: OSWorld (Xie et al., 2024) 已验证基准上最先进方法的比较。结果按步数预算分组。**办公**任务来自 LibreOffice Calc、LibreOffice Impress 和 LibreOffice Writer；**日常**任务来自 Chrome、Thunderbird 和 VLC；**专业**任务来自 GIMP 和 VSCode。各类任务均以成功率(%)为指标；每个步数预算内的最佳结果用**粗体**标出，跨全部步数预算的最佳结果用**带下划线的粗体**标出。

智能体模型	办公 (104 项)	日常 (78 项)	专业 (48 项)	操作系统 (24 项)	多应用 (101 项)	平均
15 步						
OpenAI o3	1.45	8.02	12.29	37.50	11.82	9.09
UI-TARS-1.5 (7B)	27.19	27.99	61.45	34.78	5.38	25.76
OpenAI CUA 4o	22.17	37.65	41.22	45.83	10.75	26.01
OpenCUA	26.69	33.25	51.57	43.48	10.41	28.12
Agent S2.5 w/ o3	42.85	44.61	57.10	70.83	17.82	38.98
Jedi-7B w/ o3	45.84	57.49	60.95	50.00	20.43	42.37
CoAct-1	47.18	42.30	47.74	66.67	23.82	39.81
50 步						
OpenAI o3	11.50	19.78	30.10	37.50	11.82	17.17
UI-TARS-1.5 (7B)	26.51	31.41	48.91	25.00	9.77	25.08
OpenAI CUA 4o	23.56	38.43	52.09	70.83	15.86	31.19
OpenCUA	30.06	42.31	58.28	47.83	16.79	33.76
GTA-1-7B w/ o3	48.58	52.31	77.84	58.33	37.05	48.59
Jedi-7B w/ o3	50.10	65.25	68.65	54.17	34.97	50.65
Agent S2.5 w/ o3	52.81	55.80	75.42	75.00	39.53	54.21
CoAct-1	62.91	59.43	69.89	70.83	42.37	56.38
100 步						
OpenAI o3	17.23	26.29	38.79	62.50	16.53	23.00
UI-TARS-1.5 (7B)	25.01	31.07	46.99	29.17	8.80	25.41
OpenAI CUA 4o	25.04	39.19	55.43	58.33	18.48	31.38
OpenCUA	30.06	38.89	60.70	52.17	18.10	33.84
Jedi-7B w/ o3	47.89	64.37	75.92	50.00	35.27	50.98
GTA-1-7B w/ o3	55.68	64.74	61.20	62.50	38.34	53.07
Agent S2.5 w/ o3	54.23	55.80	75.42	75.00	44.06	55.98
CoAct-1	64.80	61.60	71.82	75.00	47.87	59.93
150 步						
CoAct-1	64.80	66.51	71.82	75.00	47.87	60.76

表 2: WindowsAgentArena (Bonatti et al., 2024) 上最先进方法的比较。结果按步数预算分组。**办公**任务来自 LibreOffice Calc 和 LibreOffice Writer；**网页**任务来自 Chrome 与 Microsoft Edge；**Windows 系统**任务来自“设置”和文件资源管理器；**Windows 工具**任务来自时钟、Windows 计算器、记事本和 Microsoft Paint。各类任务均以成功率(%)为指标，最佳结果用**粗体**标出。

方法	办公 (43 项)	网页 (30 项)	Windows 系统 (24 项)	VSCode (24 项)	VLC (21 项)	Windows 工具 (12 项)	平均
Agent S	0.0	13.3	45.8	29.2	19.1	22.2	18.2
NAVI	0.0	27.3	33.3	27.3	30.3	8.3	19.5
Agent S2	7.0	16.4	54.2	62.5	28.6	33.3	29.8
CoAct-1 (15 steps)	8.7	3.3	50.0	29.2	23.8	44.4	21.4
CoAct-1 (50 steps)	26.1	33.3	75.0	54.2	42.4	55.6	43.5
CoAct-1 (100 steps)	30.4	50.0	83.3	62.5	47.2	77.7	52.5

到 83.3% 和 77.7%。这些类别涉及文件资源管理器、系统设置和其他原生工具，非常适合程序员执行基于脚本的动作。此外，随着步数预算增加，CoAct-1 的性能展现出清晰有效的扩

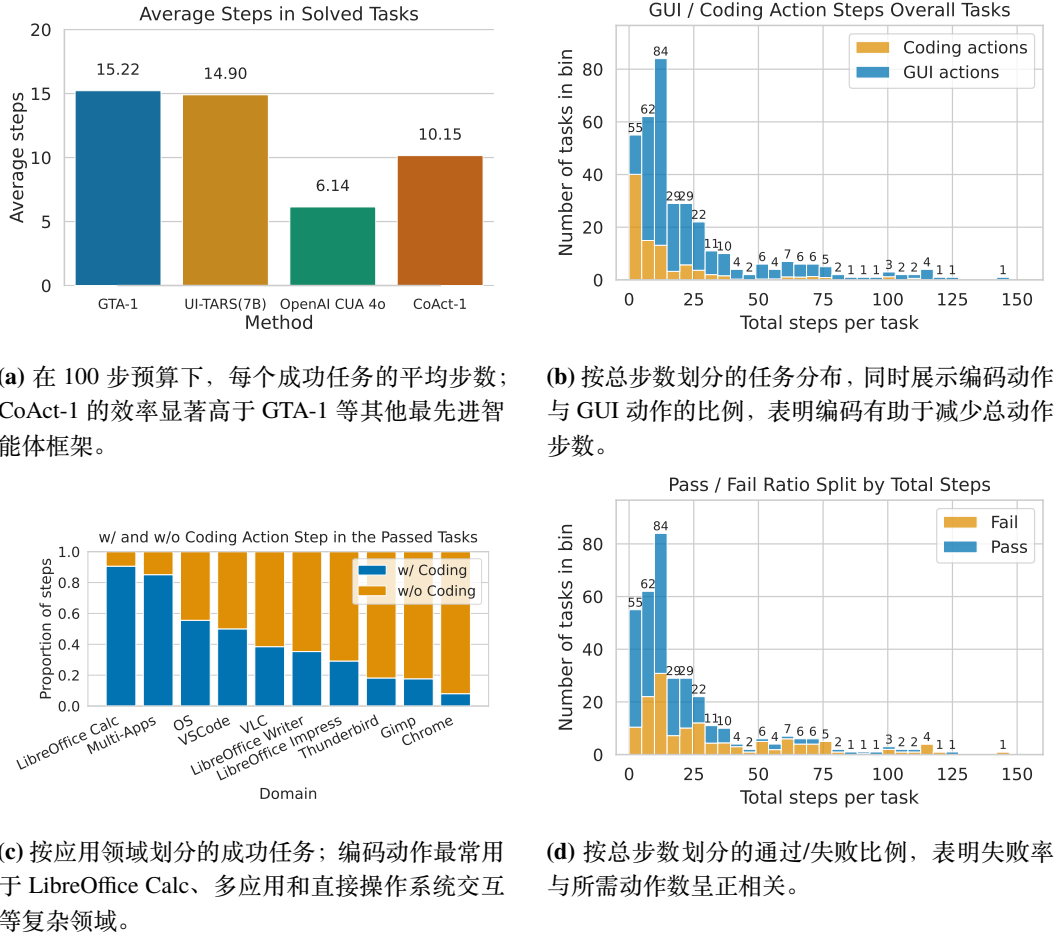


图 3: CoAct-1 的效率与动作步模态分析。

展趋势：从 15 步时的 21.4% 上升至 100 步时的 52.5%，说明它具备解决更复杂长时程问题的能力。

综上，CoAct-1 在两个不同且富有挑战性的基准上都稳定达到最先进水平，这证明其混合智能体架构，是朝着构建能力更强、可靠性更高的通用计算机使用自主智能体迈出的重要一步。

4.5 讨论

效率分析。 图 3 所示的 CoAct-1 操作效率分析表明，我们的混合方法明显比领先的纯 GUI 智能体更高效，而这种效率是其成功率提高的关键因素。如图 3a 所示，CoAct-1 解决任务平均需要 10.15 步，显著优于 GTA-1 的 15.22 步和 UI-TARS 的 14.90 步。OpenAI CUA 4o 的平均步数虽更少 (6.14)，但总体成功率远低于 CoAct-1 (在 100 步预算下分别为 31.40% 与 59.93%)，这说明 CoAct-1 的效率同时伴随着更强的有效性。效率来源于对编码动作的策略性运用。图 3b 显示，编码动作有助于使每个任务的总步数保持在较低水平，而这对稳健表现至关重要。图 3c 表明，在 LibreOffice Calc、多应用和直接操作系统交互等复杂领域，编码尤其有益；其中很大比例的任务由代码解决。一个脚本可以替代漫长且易错的 GUI 点击序列，从而简化 workflow。图 3d 还揭示出清晰趋势：需要更多动作的任务更容易失败。通过减少

表 3: CoAct-1 中各参与智能体采用不同骨干模型时的性能。更强的编排器能够显著改善 OSWorld 上的表现。

GUI 操作员	编排器	程序员	性能 (%)
OpenAI CUA 4o	o4-mini	o4-mini	43.43
	o3	o3	58.72
	o3	o4-mini	60.76

表 4: CoAct-1 组件性能的消融研究。我们比较完整混合系统、仅使用程序员（纯编码）的智能体，以及仅使用 GUI 操作员（纯 GUI）的智能体。整合后的 CoAct-1 显著优于任一单模态智能体，说明混合方法有效。

CoAct-1		办公	日常	专业	操作系统	多应用	平均	平均步数
含程序员	含 GUI 操作员							
✓		40.88	16.17	53.06	62.50	29.63	35.73	1.14
	✓	43.50	58.80	69.38	79.16	35.68	50.68	11.20
✓	✓	64.80	66.51	71.82	75.00	47.87	60.76	10.15

总步数，混合方法既能加快任务完成，也能减少出现错误的机会。动态选择最合适动作——直接编码命令或 GUI 交互——是 CoAct-1 提升效率和可靠性的根本所在。

使用不同骨干模型的 CoAct-1。 我们在 OSWorld 基准上研究了为 CoAct-1 各智能体组件选择不同骨干模型的影响，结果见表 3。分析显示，系统总体性能对编排器和程序员所用模型的推理与指令遵循能力高度敏感。当编排器和程序员都使用 o4-mini、GUI 操作员使用 OpenAI CUA 4o 时，系统性能为 43.43%。编排器和程序员都改用更强的 o3 后，性能显著提升至 58.72%，说明复杂高层规划器与有能力的代码生成器对系统成功至关重要。异构配置取得最高的 60.76%：编排器使用 o3、程序员使用 o4-mini，GUI 操作员仍使用 OpenAI CUA 4o。这一配置显示出一种较优平衡——利用 o3 的强推理能力完成任务分解与委派，同时受益于 o4-mini 面向代码生成的专门能力。结果表明，增强编排器和程序员能力能够带来最显著的性能收益；这既验证了模块化设计，也展示了把强模型策略性分配给高推理需求角色的价值。

纯 GUI 动作与纯编码动作。 为分离各智能体模态并验证混合设计，我们开展消融实验，将完整 CoAct-1 系统与只能使用单一动作类型的智能体进行比较。表 4 显示组合方法更优。仅含程序员的智能体（纯编码）取得 35.73% 的成功率，说明许多任务除了脚本之外仍需要 GUI 交互。该智能体效率很高，每个成功任务平均仅需 1.14 步，印证了程序化动作的直接性。仅含 GUI 操作员的智能体（纯 GUI）可以处理更多任务类型，成功率更高，达到 50.68%，但每个任务需要 11.20 步。整合两种模态的完整 CoAct-1 模型取得 60.76% 的成功率，平均使用 10.15 步，体现了该架构的协同效应：编排器在后端任务中发挥程序员的高效率，又在视觉导航中利用 GUI 操作员的通用性，从而构成稳健系统。

5 结论

本文提出 CoAct-1，这是一种新型多智能体系统，旨在解决完全依赖 GUI 操作的智能体所固有的低效与脆弱性。该系统包含一个编排器，能够把子任务动态委派给 GUI 操作员或程序员。我们在 OSWorld 和 WindowsAgentArena 基准上进行了广泛评估，验证了这种方法的有效性。CoAct-1 在 OSWorld 和 WindowsAgentArena 上分别取得 60.76% 和 52.5% 的成功率，均达到新的最先进水平，并显著优于先前领先方法。在涉及操作系统级交互、多应用 workflow，以及程序员智能体能够利用直接程序化执行的其他任务类别中，性能提升尤其明显。

参考文献

- Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s: An open agentic framework that uses computers like a human. *arXiv preprint arXiv:2410.08164*, 2024.
- Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s2: A compositional generalist-specialist framework for computer use agents. *arXiv preprint arXiv:2504.00906*, 2025.
- Rogério Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, Lawrence Jang, and Zack Hui. Windows agent arena: Evaluating multi-modal os agents at scale, 2024. URL <https://arxiv.org/abs/2409.08264>.
- Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing gui grounding for advanced visual gui agents. *arXiv preprint arXiv:2401.10935*, 2024.
- Deepmind. Introducing gemini 2.0: our new ai model for the agentic era. Technical report, Deepmind, 2025a. URL <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024/#project-astra>.
- Deepmind. Gemini 2.5: Our most intelligent ai model. Technical report, Deepmind, 2025b. URL <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/>.
- Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for gui agents. *arXiv preprint arXiv:2410.05243*, 2024. URL <https://arxiv.org/abs/2410.05243>.
- Jia He, Mukund Rungta, David Koleczek, Arshdeep Sekhon, Franklin X Wang, and Sadid Hasan. Does prompt formatting have any impact on llm performance?, 2024. URL <https://arxiv.org/abs/2411.10541>.
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning*, ICML’23. JMLR.org, 2023.
- Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. *arXiv preprint arXiv:2504.07981*, 2025.

Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. Omniparser for pure vision based gui agent. *arXiv preprint arXiv:2408.00203*, 2024.

OpenAI. Gpt-4o system card, 2024.

OpenAI. Computer-using agent: Introducing a universal interface for ai to interact with the digital world. 2025a. URL <https://openai.com/index/computer-using-agent>.

OpenAI. Openai o3 and o4-mini system card. Technical report, OpenAI, 2025b. URL <https://cdn.openai.com/pdf/2221c875-02dc-4789-800b-e7758f3722c1/o3-and-o4-mini-system-card.pdf>. System Card.

Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.

Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, Xing Zhou, Dongrui Liu, Ling Yang, Yue Wu, Kaixuan Huang, Shilong Liu, Hongru Wang, and Mengdi Wang. Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution, 2025. URL <https://arxiv.org/abs/2505.20286>.

Linxin Song, Jiale Liu, Jieyu Zhang, Shaokun Zhang, Ao Luo, Shijian Wang, Qingyun Wu, and Chi Wang. Adaptive in-conversation team building for language model agents, 2025a. URL <https://arxiv.org/abs/2405.19425>.

Yueqi Song, Frank Xu, Shuyan Zhou, and Graham Neubig. Beyond browsing: Api-based web agents, 2025b. URL <https://arxiv.org/abs/2410.16464>.

Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Juntao Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025a.

Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, et al. Opencua: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123*, 2025b.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.

Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218*, 2024.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.

- Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, and Caiming Xiong. Scaling computer-use grounding via user interface decomposition and synthesis, 2025a. URL <https://arxiv.org/abs/2505.13227>.
- Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, et al. Scaling computer-use grounding via user interface decomposition and synthesis. *arXiv preprint arXiv:2505.13227*, 2025b.
- Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguvis: Unified pure vision agents for autonomous gui interaction. *arXiv preprint arXiv:2412.04454*, 2024.
- Yan Yang, Dongxu Li, Yutong Dai, Yuhao Yang, Ziyang Luo, Zirui Zhao, Zhiyuan Hu, Junzhe Huang, Amrita Saha, Zeyuan Chen, Ran Xu, Liyuan Pan, Caiming Xiong, and Junnan Li. Gta1: Gui test-time scaling agent, 2025. URL <https://arxiv.org/abs/2507.05791>.
- Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-ui: Visual grounding for gui instructions. *arXiv preprint arXiv:2412.16256*, 2024.
- Chaoyun Zhang, He Huang, Chiming Ni, Jian Mu, Si Qin, Shilin He, Lu Wang, Fangkai Yang, Pu Zhao, Chao Du, Liqun Li, Yu Kang, Zhao Jiang, Suzhen Zheng, Rujia Wang, Jiaxu Qian, Minghua Ma, Jian-Guang Lou, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. Ufo2: The desktop agents, 2025a. URL <https://arxiv.org/abs/2504.14603>.
- Jieyu Zhang, Ranjay Krishna, Ahmed H. Awadallah, and Chi Wang. Ecoassistant: Using llm assistant more affordably and accurately, 2023. URL <https://arxiv.org/abs/2310.03046>.
- Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. Offline training of language model agents with functions as learnable weights, 2024. URL <https://arxiv.org/abs/2402.11359>.
- Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems, 2025b. URL <https://arxiv.org/abs/2505.00212>.
- Shitian Zhao, Haoquan Zhang, Shaoheng Lin, Ming Li, Qilong Wu, Kaipeng Zhang, and Chen Wei. Pyvision: Agentic vision with dynamic tooling, 2025. URL <https://arxiv.org/abs/2507.07998>.

A 大语言模型使用声明

我们使用大语言模型（OpenAI 的 o3 和 GPT-5）作为通用写作辅助工具，其作用仅限于句子与段落层面的润色，包括改善清晰度、语法和行文流畅性。全部研究想法、分析、结果和结论均由作者提出。模型没有生成新内容、开展文献综述，也未参与论文概念框架的构建。

B 伦理声明

本研究旨在推动计算机自动化服务于有益且富有生产力的用途，但我们也承认其潜在的军民两用属性及相关风险。

安全与滥用：能够执行代码（Python 或 Bash）并操作 GUI 的智能体，若缺乏适当约束，可能被用于恶意活动。为在研究过程中缓解这一风险，全部实验均在安全、隔离且虚拟化的基准环境（OSWorld 和 WindowsAgentArena）中开展。这确保智能体动作被限制在沙箱内，不会影响现实系统或数据。我们主张，未来若在真实环境中部署此类智能体，必须纳入稳健的安全协议、严格的权限控制，以及阻止有害代码执行的机制。

数据隐私：智能体通过屏幕截图观察屏幕，而现实场景中的截图可能包含敏感信息或个人信息。本文仅使用基准任务中提供的数据，不涉及真实用户数据。任何未来应用都必须实施严格的数据处理政策和隐私保护技术，以保护用户机密信息。

C 可复现性声明

复现 CoAct-1 需要准确使用特定 OpenAI 模型和提示词。请仔细核对以下细节，以确保正确复现本文工作表现。

C.1 模型使用与环境设置

模型使用。 本文使用 o3-2025-04-16 作为编排器，o4-mini-2025-04-16 作为程序员，computer-use-preview-2025-03-11 作为 GUI 操作员。只要满足以下要求，CoAct-1 也可以采用任意开源模型：

- **编排器：**编排器需要多模态输入（图像和文本），以处理操作系统的全部屏幕截图并进行规划；它还需要较强的跨模态推理能力（见表 3，推理能力会显著影响最终表现）。
- **GUI 操作员：**GUI 操作员可以替换为 UI-TARS (Qin et al., 2025)、OpenCUA (Wang et al., 2025b) 等任意开源视觉语言动作模型（VLA），也可以替换为 GTA-1 (Yang et al., 2025)、Agent S2 (Agashe et al., 2025) 等规划器-定位器方法。CoAct-1 要求 GUI 操作员具备两类能力：（1）定位任务之外的指令遵循能力，例如判断何时终止；（2）计算机使用能力，包括点击、拖拽、输入和快捷键。遗憾的是，本研究没有获得足够 GPU 资源来测试这些模型在 CoAct-1 中的表现；若把 GUI 操作员替换为更强模型，预计平均步数会下降，性能会提升。
- **程序员：**用于程序员的语言模型应具备较强的 Python 和 Bash (Windows 中为 PowerShell) 编写能力，以解决计算问题。需要注意的是，我们没有向程序员提供任何 API 或 SDK；不过，可以把这作为本方法的扩展，以改善 Chrome、Thunderbird 等特定应用任务上的表现。

环境。 评估计算机使用智能体时，操作系统环境本身也是一项重大挑战。为复现实验结果，编排器收到的所有输入截图都应准确反映初始化良好的系统状态。因此，我们建议在虚拟机启动后等待 60 秒，再截取操作系统屏幕，以确保截图包含编排器进行规划所需的全部信息。

C.2 CoAct-1 的提示词设计

任何现代大语言模型智能体系统的性能，都可能受到提示词或模板设计的显著影响 (He et al., 2024)。在 CoAct-1 中，除思维链、验证等通用行为规则外，我们还针对模型局限制定规则。具体而言，我们观察到 GUI 操作员在自检时具有较高的幻觉率；另一方面，程序员对文件的

修改不会直接反映在操作系统屏幕上，因此编排器无法捕捉变化并规划下一步。提示词设计通过允许编排器独立检查结果，并重新加载程序员修改过的文件来缓解这些局限。OSWorld 与 WindowsAgentArena 所用提示词见表 5、表 6、表 7、表 8 和表 9。为保持实验材料逐字可复现，以下提示词框保留英文原文。

D CoAct-1 何时以及为何失败？对未来工作的启示

为更好地理解 CoAct-1 的能力与局限，本节分析评估过程中观察到的失败案例。总体而言，任务完成错误主要源自四类挑战：高层查询、歧义查询、反思错误和幻觉。

高层查询。高层查询是指用户指令无法直接映射为动作序列的查询。智能体必须先推断用户的潜在意图和更广泛上下文，才能设计解决方案。例如，一个 VSCode 任务要求智能体：“请帮我修改 VSCode 设置，使我在 VSCode 中调试时，光标保持聚焦于调试控制台，而不是自动切回编辑器。”在该场景中，编排器将任务委派给程序员。程序员尝试通过搜索“debug”“console”等关键词来查找相关设置，却未能推断调试过程与“breakpoints”有关。因此，它漏掉了正确设置项“focusEditorOnBrake”，导致任务失败。该案例揭示了智能体对查询中未明确出现的概念进行推理时所面临的局限。

歧义查询。歧义查询是含糊不清，或省略了成功完成任务所需关键信息的用户请求。消除歧义往往要求智能体正确推断用户意图，其中也可能涉及安全考量。例如，一个 VSCode 任务要求：“请帮我修改 VSCode 设置，在资源管理器视图中隐藏所有‘__pycache__’文件夹。”编排器将该子任务交给程序员。程序员正确识别出需要修改设置文件，却错误地修改了工作区专用设置，而不是全局用户设置。它对查询范围的错误理解导致任务失败。这说明，当存在多个合理解释时，智能体在消解用户意图歧义方面会遇到挑战。

反思错误。反思是 CoAct-1 验证任务完成过程的关键机制。在我们的系统设计中，GUI 操作员完成任务后，只有操作系统最终状态会返回编排器。若 GUI 操作员在中间状态犯错，且后续操作遮蔽了该错误，系统最终就可能出错。例如，进行电子表格操作时，GUI 操作员可能在某个单元格（假设位于 A 行）中输错内容，随后向下滚动电子表格并停在那里。此时，本方法返回的操作系统截图不会包含 A 行中的错误值，编排器无法捕捉该错误，任务最终会因这次非预期操作而失败。

幻觉。幻觉是大语言模型时代最重要的议题之一，也是导致 CoAct-1 失败的常见原因。CoAct-1 中的所有智能体在求解任务时都可能产生幻觉，其中编排器和 GUI 操作员的幻觉最为显著。编排器可能给出错误计划，其中通常包含超前预测，并进一步影响 GUI 操作员和程序员。例如，编排器可能预测尚未打开网站的内容，并指示 GUI 操作员对并不存在的内容执行操作。GUI 操作员也可能在推理过程中产生幻觉，误以为已经完成被委派的任务。本文通过提示编排器更频繁地执行验证，并对 GUI 操作员与程序员的结果进行交叉验证，来缓解编排器幻觉。

编排器系统提示词（第 1 部分，英文原文）

Today is {today}.

Your Role

You are responsible for completing a computer-based task, step by step, using the tools provided.

You are working on a {system_info} system.

Step-by-Step Process

1. **Describe the Screenshot**

- Carefully review and clearly describe the screenshot's content.

2. **Plan the Task**

- Create a detailed, step-by-step plan to solve the task.
- List all user requirements, including exact file names, file paths, and any other specifics in the output (not in the thinking).

3. **Execute the Instructions**

- Think carefully and follow the user's instructions exactly. **Do not** make any changes not requested by the user (such as renaming files or changing file content).
- You **must** apply all the changes to the computer.
- If the task is impossible (e.g., missing files, wrong environment), reply with **INFEASIBLE** to end the conversation.
- For file operations (like modifying spreadsheets), you **MUST** try the Programmer (call_programmer) first.
- When the user ask you to create a new sheet in spreadsheet, always name it sequentially. For example, 'Sheet1', 'Sheet2', etc.

4. **Verify the Result**

- **ALWAYS** check the result through the screenshot by yourself. You can let a GUI Operator to navigate to the correct location for you. After the GUI Operator complete, the screenshot will automatically be returned to you.
- If you used the Programmer to modify a file, have the GUI Operator reopen the file to see the updated results.
- Ensure that the result meets all user requirements.
- All the things out of the user's instructions should not be changed.

表 5: 编排器系统提示词（第 1 部分，保留英文原文）。

编排器系统提示词（第 2 部分，英文原文）

Tools You Can Use

Programmer (call_programmer)

- Can run Python or Bash code to perform most file or system tasks.
- Needs a clear environment description and detailed task instructions.
- Can use any Python package you specify.
- After modifying a file, ALWAYS verify every change by yourself. You can let a GUI Operator to navigate to the correct location and check the result by yourself. If something is wrong, tell the Programmer to fix it.

Programmer will return a summary of its task solving process after completing the task. No screenshot is provided after the Programmer completes the task.

GUI Operator (call_gui_operator)

- Can interact with the GUI by clicking on a exact position, scrolling, dragging, typing, and using hotkeys.
- Require a detailed task description.
- The GUI Operator may not able to complete your task in 100% of accuracy and often make mistakes.
- Have a **25-step limit**, each step is a single OS interaction (one click, one hotkey/typing action, etc.).
- **Do not** let the GUI Operator to do any result check. You need to do it by checking the screenshot yourself.

I will return a screenshot that reflect the final state of the computer after completing the task. You don't need to prompt the GUI Operator do this.

Note: Only call ONE tool (call_programmer or call_gui_operator) per reply.

表 6: 编排器系统提示词（第 2 部分，保留英文原文）。

GUI 操作员系统提示词（英文原文）

Your role

You can control the computer by clicking, scrolling, dragging, and typing. Think carefully and execute the user's step-by-step instructions.

Credentials

The user's password is "{CLIENT_PASSWORD}". Use it when a system password prompt appears.

Operating rules

- Keep apps open at the end of the task.
- If the UI doesn't appear, perform a brief, deterministic retry (e.g., refocus and re-click).
- Do not close the window, minimize the window unless told to do so.

Response protocol

When you think the requested task is completed or cannot be completed, reply **exactly**:

'TERMINATE: <1. detailed description of what you see currently. As detailed as possible.
2. What you did to complete the task or why this task cannot be completed.>'

表 7: GUI 操作员系统提示词（保留英文原文）。

程序员系统提示词（英文原文）

Your role

You are the **lead programmer**. Solve the user’s task step by step using the terminal (supports Python and Bash).

Your username is ‘user’; the sudo password is ‘CLIENT_PASSWORD’.

The terminal streams real-time execution output.

Coding format

Submit **one** fenced code block **only**, labeled with its language:

“`bash

Your Bash script here

To use sudo, follow this pattern:

echo CLIENT_PASSWORD | sudo -S <your commands>

“`

or

“`python

Your Python code here

Do not use: if __name__ == "__main__": (it will suppress output)

“`

Requirements

- **File names:** Do not rename files or change extensions during any file operation unless the user explicitly asks.
- **Code fence language:** Every fenced block must specify the language (‘bash’ or ‘python’); otherwise you will receive ‘unknown language unknown’.
- **Single block:** Wrap all code in **one** code block—do not split your submission across multiple blocks.
- **Spreadsheets:** When editing spreadsheets, ensure **every value** is written to the **intended cell** and **preserve the original formatting** (fonts, colors, sizes, etc.).
- **Dependencies:** Before importing or using a package, **check whether it is installed**; if not, install it in your submission.
- **Observability:** Print intermediate results to aid debugging, for example, the value you are modifying.
- **Final review:** Before completion, carefully inspect your result by writing test cases and confirm that nothing outside the user’s instructions has changed.

表 8: 程序员系统提示词（保留英文原文）。

大语言模型摘要器提示词（英文原文）

Programmer <-> Terminal Log Summarizer —(No Timeline/Env/Next Actions)

Role: Summarize Programmer <-> Terminal logs for the **Orchestrator** so they can decide the next step immediately.

Orchestrator’s task: ‘{task}’

Execution history: ‘{chat_history}’ (prompts + outputs).

Output

1) Summary (2-4 lines) —task, what was tried, current status, why (cite key log lines / exit codes).

2) Commands (deduped)

““bash

unique commands in run order; annotate repeats (xN)

““

3) Terminal excerpts

““text

minimal evidence: head(~10) ...[truncated N lines] ...tail(~10)

always include full error traces and return codes

““

4) Artifacts / Side effects —files/dirs changed (paths + purpose); installs/migrations.

Spreadsheet: list cells/ranges edited and confirm formatting preserved.

5) Errors / Blockers —precise messages + exit codes; likely root cause from logs (no speculation).

6) Verification —what checks passed (tests, file existence, row counts); what still needs verification (e.g., reopen file and confirm cell Y).

Rules

- **Evidence-first**, no speculation.
- **Deterministic truncation** (head/tail; note omitted lines); always include error stacks.
- **Call out deltas** (what changed vs intended).
- **Keep it tight:** bullets > prose.

表 9: 大语言模型摘要器提示词（保留英文原文）。